Lagom-style service descriptor for announce-management

Implementation plan for worktree `service-descriptor` (branch `feat/service-descriptor`), written 2026-10-24 before any code change, as the repository process rule asks. Source of the task: the block "Worktree: service-descriptor" in CLAUDE.md (2026-10-22). Status: **awaiting approval**.

1. Goal in one paragraph

The Lagom split is in place except for one piece: there is no shared `@HttpExchange` service descriptor. messaging carries a private copy of the announce read API. This task publishes the read slice of the announce API (`view`, `browse`) as a descriptor in `announce-management-api`, lets `AnnounceController` implement it server-side (Spring Framework 6.2), moves caller identity out of the method signatures into a WebFilter plus Reactor context, deletes messaging's private copy, adds the same pattern to the framework sample, and documents it. The relay stays the only Kafka producer; nothing in `cqrs-reactive` main code changes.

2. Survey facts that shape the plan

- `AnnounceController`: class-level `@RequestMapping("/api/announces")`; every method takes bare `HttpHeaders`; `view` and `browse` resolve the viewer with `Callers.viewer(headers)`. `browse` filters: kind, category (list), brand, seller, region, q, saleType, status, page=1, size=20, sort=newest.
- `AnnounceDtos` holds request and response records with the domain enums `Kind`, `AnnounceStatus`, `SaleType`, `Condition`, `ContractType`, plus `toDetails()` and `of()` logic. It is also used by `AnnounceAdminController` (response factories, PageResponse) and `BrandController` (PageResponse). No test imports it.
- messaging: `AnnounceLookup.find(id)` returns `Mono<AnnounceRef>`; `Conversations.openListing` uses only `sellerId()`. Two test stubs build `AnnounceRef`. `application-test.yml` already pins `cqrs.api.clients.announce-management.base-url`; production config sets `service-token: true`.
- Framework: `ServiceTokenWebFilter` is ordered `MIN_VALUE + 200`; `ServiceTokenAutoConfiguration` also exposes a `ServiceTokenMinter`. `HttpExchangeClients` pins a base URL from `CqrsProperties.Api` and adds the token filter when the client opts in. `EdgeRequests` (test kit) signs test requests.
- Sample: `ListingController` has class `@RequestMapping("/listings")`, nested `CreateRequest`, and `get` returns `Mono<ResponseEntity<Summary>>`. `SampleEndToEndTest` runs with the service token off.
- `announce-management-api` depends on `cqrs-reactive` only, which brings `spring-webflux` (and so `@HttpExchange`, `@RequestParam`, `Mono`) transitively. The root pom already manages the artifact.
- Framework 6.2 rule: a class or a method must not carry both `@RequestMapping`-family and `@HttpExchange`-family annotations; mixing across methods of one class is legal. A class-level `@HttpExchange` prefix combines with a method-level `@PostMapping`.
- ReportLab 5.0.1 is installed for the session report. No announce-management or messaging instance is running on the machine.

3. Two readings I had to make

1 **Extra api enums.** The block names `AnnounceKindV1`, `AnnounceStatusV1`, `SaleTypeV1`, but the response records also carry `Condition` and `ContractType`. Decision 5 forbids domain imports in the api module, so I add `ConditionV1` and `ContractTypeV1` with the same constant names.

2 **Replacement for AnnounceRef.** Deleting it leaves `AnnounceLookup` without a return type. I give the port a nested record `AnnounceLookup.AnnounceFacts(id, sellerId, title, status)` and the api-backed lookup maps `AnnounceResponse` into it. The alternative, returning the 18-field `AnnounceResponse` from the port, makes the two test stubs ugly and couples the application layer to the wire type.

4. Steps

Step 1. Descriptor and DTOs in announce-management-api (package `org.linlinjava.announce.goods.api`)

- `AnnounceApi`: `@HttpExchange("/api/announces")`; `@GetExchange("/{id}") Mono<AnnounceResponse> view(@PathVariable String id)`; `@GetExchange Mono<PageResponse<AnnounceSummaryResponse>> browse(...)` with every filter `@RequestParam(required = false)` (kind, category, brand, seller, region, q, saleType, status as V1 enums where applicable) and `page`, `size`, `sort` with defaults 1, 20, newest.
- `AnnounceApiDtos`: `AnnounceResponse`, `AnnounceSummaryResponse`, `PageResponse`, `LocationDto`, `GoodsDto`, `JobDto`, `EventDto`, `VariantDto`, `AttributeDto`, `SpecificationDto` as logic-free records with the same field names and JSON shape as today.
- Enums `AnnounceKindV1`, `AnnounceStatusV1`, `SaleTypeV1`, `ConditionV1`, `ContractTypeV1`, string-serialised, same names as the domain enums.
- No pom change in the api module.

Step 2. Mapper in announce-management

- `AnnounceApiMapper` next to the controller: `response(Announce)`, `summary(AnnounceSummary)`, `toContent(AnnounceRequest)`, `toDetails(AnnounceRequest)`, and enum conversions by name in both directions.
- `AnnounceDtos` shrinks to `AnnounceRequest` and `PublishRequest` (the write side stays local, out of scope for the interface).
- `AnnounceAdminController` and `BrandController` switch to the api records and the mapper.

Step 3. Caller filter (`goods.interfaces.rest`)

- `CallerContextFilter`, a `WebFilter` at order `MIN_VALUE + 300` (after `ServiceTokenWebFilter`), runs the framework `Callers.caller(headers)` once and writes the result into the Reactor context under one key.
- A parse failure (non-numeric `X-User-Id` gives 400, unknown `X-User-Type` gives 403) is stored, not thrown in the filter, and re-raised when a route reads the caller, so the existing problem-detail advice keeps producing the same answers.
- The context `Callers` gains no-arg overloads: `member()` returning `Mono<SellerId>` (401/403 as today), `viewer()` returning `Mono<SellerId>` that is empty for a visitor and for staff, `staff(permission)` returning `Mono<Integer>`. The `HttpHeaders` overloads stay for the admin and listing controllers.
- Not added to messaging or order.

Step 4. AnnounceController implements AnnounceApi

- Drop the class-level `@RequestMapping` and the two `@GetMapping` lines; the interface carries them. The write methods keep `@PostMapping` / `@PutMapping` and `HttpHeaders` exactly as today.
- `view` and `browse` read the viewer from the context and map the V1 filter enums to the domain.
- `AnnounceControllerTest` imports the filter; its visitor-versus-owner tests (drafts visible to the owner only) stay as the proof of the caller filter.
- New `AnnounceApiRoundTripTest`: a `@WebFluxTest` with the service token auto-configuration imported and enforced, an `HttpExchangeClients` built over an `HttpHandlerConnector` to the test context with

the base URL pinned and `service-token=true`. Asserts that `view` and `browse` through the client equal the JSON the controller serves directly, and that a client without the token gets 401.

Step 5. messaging

- Delete `infrastructure/announce/AnnounceApi` and `application/AnnounceRef`.
- `AnnounceLookupConfig` creates `clients.create(org.linlinjava.announce.goods.api.AnnounceApi.class, "announce-management")` and maps `AnnounceResponse` to `AnnounceLookup.AnnounceFacts`; 404 stays an empty Mono.
- Add `announce-management-api` to `messaging/pom.xml`; update the two test stubs; keep `service-token=true` and the `messaging.announce-lookup=stub` switch.

Step 6. Framework sample (`cqrs-reactive-test`, package `sample`)

- `ListingApi`: `@HttpExchange("/listings")`, `view` and `post` (`CreateRequest` moves into the interface; `X-Command-Id` as `@RequestHeader(required = false)`). `ListingController implements ListingApi`, class mapping dropped, `@ResponseStatus(CREATED)` stays on the implementation.
- `SampleEndToEndTest` turns the service token on with `EdgeRequests`, signs its `WebTestClient`, and adds one round trip through an `HttpExchangeClients` built in the test with the local port pinned and the token on, comparing the client's record to the served JSON.

Step 7. Docs and report

- `doc/cqrs-reactive.md` §4 gains "Implementing the descriptor" (server-side `@HttpExchange`, client-legal parameters, caller filter, no mixing on one method, the relay is the only producer) and the Lagom table gains the row "service implementation implements the descriptor".
- `doc/announce-management-context.md` records the api module layout.
- Session report PDF in `doc/`; memory file updated with the status.

5. Files touched

Module	New	Changed	Deleted
announce-management-api	AnnounceApi, AnnounceApiDtos, five V1 enums	—	—
announce-management	AnnounceApiMapper, CallerContextFilter, AnnounceApiRoundTripTest	AnnounceController, AnnounceDtos, Callers, AnnounceAdminController, BrandController, AnnounceControllerTest	—
messaging	—	AnnounceLookup, AnnounceLookupConfig, pom.xml, MessagingEndToEndTest, StatementCostProbeTest	infrastructure/announce/AnnounceApi, application/AnnounceRef
cqrs-reactive-test	sample/ListingApi	ListingController, SampleEndToEndTest	—
doc	service-descriptor-report.pdf	cqrs-reactive.md, announce-management-context.md	—

6. Builds and verification

- `mvn -pl cqrs-reactive,cqrs-reactive-test test` (Testcontainers only).
- `mvn -pl cqrs-reactive,announce-management-api,announce-management -P '!webapp' test` in one reactor, so a stale `cqrs-reactive` jar in the shared `~/.m2` cannot leak in; containers plus the live MySQL as today.

- `mvn -pl announce-management-api,messaging test` after checking that no messaging instance holds the live database (a running instance stalls `MessagingEndToEndTest`).
- Acceptance checks from the block: messaging compiles with no local descriptor; the round-trip tests pass; a request without `X-User-Id` sees only PUBLISHED announces and one with it sees its own drafts; §4 of the framework doc updated.
- Manual check (member opens a listing conversation in the customer SPA, messaging log shows the call through the shared descriptor) is run only if the stack is started in this session; otherwise it is recorded as not run.

7. Out of scope (unchanged from the block)

Member writes on the interface, admin and listing controllers on the caller filter, descriptors for order and messaging, DTO versioning beyond the V1 enums, any change in `cqrs-reactive` main code.